Supplementary Materials

Noise Decorrelation as a Hypothetical Mechanism for
Phase-Specific Neurometabolic Outcomes in HIV Infection:
A Computational Framework

A.C. Demidont, DO
Independent Researcher, Nyx Dynamics
268 Post Rd, Suite 200, Fairfield, CT 06428
Email: acdemidont@nyxdynamics.org
ORCID: 0000-0002-9216-8569

January 2026

Contents

1 Extended Methods **3**
1.1 Hierarchical Bayesian Model Specification . . . 3
1.1.1 Likelihood . . . 3
1.1.2 Expected Value . . . 3
1.1.3 Protection Factor . . . 3
1.1.4 Prior Specifications . . . 3
1.2 MCMC Sampling Procedure . . . 4
1.3 Convergence Diagnostics . . . 4
1.4 ART Era Classification . . . 4

2 Sensitivity Analyses **5**
2.1 Prior Sensitivity Analysis . . . 5
2.2 Leave-One-Study-Out Analysis . . . 5
2.3 Subgroup Analysis by Brain Region . . . 5
2.4 Model Comparison (WAIC) . . . 6

3 Supplementary Figures **7**
3.1 Prior Sensitivity Analysis . . . 7
3.2 Power Analysis . . . 7
3.3 Leave-One-Study-Out Analysis . . . 7
3.4 Subgroup Analysis by Brain Region . . . 7
3.5 Model Version Comparison . . . 7

4 Complete Data Tables **8**
4.1 Full Study Cohort Characteristics . . . 8
4.2 Complete Posterior Parameter Estimates . . . 9
4.3 Enhanced Statistical Summary . . . 9

5 Code Repository **10**
5.1 Core Bayesian Model (v3.6) 10
5.2 Enzyme Kinetics Validation Model (v4.0) 13
5.3 Statistical Improvements Module 16
5.4 Data Loader Utilities 18
5.5 Software Environment 21
5.6 Reproducibility 21

1 Extended Methods

1.1 Hierarchical Bayesian Model Specification

The hierarchical Bayesian model implements a mechanistic framework linking environmental noise correlation length (ξ) to neuronal metabolic protection. The full model specification is:

1.1.1 Likelihood

$$\mathrm{NAA}_{ij} \sim \mathcal{N}(\mu_{ij}, \sigma^2_{\mathrm{NAA}}) \tag{1}$$

where i indexes studies and j indexes observations within studies.

1.1.2 Expected Value

$$\mu_{ij} = \mathrm{NAA}_{\mathrm{base}} \cdot \Pi_\xi(\mathrm{phase}_j) \cdot (1 + \epsilon_{\mathrm{era}} + \epsilon_{\mathrm{study},i}) \tag{2}$$

1.1.3 Protection Factor

The protection factor implements the core mechanistic hypothesis:

$$\Pi_\xi = \left(\frac{\xi_{\mathrm{ref}}}{\xi}\right)^{\beta_\xi} \tag{3}$$

where:

- $\xi_{\mathrm{ref}} = 0.66$ nm (reference correlation length)
- β_ξ = protection scaling exponent (inferred)
- ξ_{phase} = phase-specific correlation length (inferred for acute, chronic, control)

1.1.4 Prior Specifications

Table 1: Prior distributions for model parameters

Parameter	Prior	Justification
$\log(\xi_{\mathrm{control}})$	$\mathcal{N}(\log(0.66), 0.15)$	Weakly informative around reference
$\log(\xi_{\mathrm{acute}})$	$\mathcal{N}(\log(0.61), 0.15)$	Centered below control
$\log(\xi_{\mathrm{chronic}})$	$\mathcal{N}(\log(0.80), 0.15)$	Centered above control
β_ξ	$\mathrm{TruncNormal}(1.89, 0.25; \mathrm{lower} = 0)$	Informed by prior theoretical work
$\mathrm{NAA}_{\mathrm{base}}$	$\mathcal{N}(2.2, 0.5)$	Literature-based healthy NAA/Cr
σ_{study}	$\mathrm{HalfNormal}(0.10)$	Study-level heterogeneity
σ_{NAA}	$\mathrm{HalfNormal}(0.20)$	Observation noise

1.2 MCMC Sampling Procedure

Inference was performed using PyMC (version 5.x) with the following settings:

- Sampler: NUTS (No-U-Turn Sampler)
- Chains: 4
- Draws per chain: 1,500
- Tune (warmup): 1,000
- Target acceptance rate: 0.92
- Random seed: 2025 (for reproducibility)

1.3 Convergence Diagnostics

Convergence was assessed using:

1. $\hat{R}$ **(Gelman-Rubin statistic)**: All parameters achieved $\hat{R} < 1.02$
2. **Effective Sample Size (ESS)**: Range 142–452 across key parameters
3. **Divergent transitions**: 0 across all chains
4. **Visual inspection**: Trace plots and rank plots showed adequate mixing

1.4 ART Era Classification

Studies were classified by antiretroviral therapy (ART) era:

- **Pre-modern** (≤ 2006): Before widespread cART with integrase inhibitors
- **Post-modern** (≥ 2007): Modern cART era with improved CNS penetration

ART era was included as an additive covariate effect, not altering the core $\xi \to \Pi_\xi$ mechanism.

2 Sensitivity Analyses

2.1 Prior Sensitivity Analysis

To assess robustness to prior specification, the model was re-run under four alternative prior scenarios:

Table 2: Prior sensitivity analysis results

Prior Type	ξ_{acute} (nm)	SD	$P(\xi_a < \xi_c)$	Interpretation
Weakly informative (default)	0.425	0.065	0.9997	Decisive
Flat/Uniform	0.431	0.072	0.9989	Decisive
Literature-based (tight)	0.419	0.058	0.9999	Decisive
Skeptical (narrow)	0.440	0.051	0.9982	Decisive

Conclusion: Results are robust to prior specification. The range of ξ_{acute} estimates across priors is < 0.025 nm ($< 6\%$ variation), and all scenarios yield decisive evidence for the protective effect ($P > 0.998$).

2.2 Leave-One-Study-Out Analysis

To ensure no single study drives conclusions, the analysis was repeated excluding each cohort:

Table 3: Leave-one-study-out sensitivity results

Dataset	ξ_{acute} (nm)	SD	$P(\xi_a < \xi_c)$
Full data	0.425	0.065	0.9997
− Sailasuta 2016	0.431	0.072	0.9994
− Young 2014	0.419	0.069	0.9996
− Sailasuta 2012	0.427	0.066	0.9995
− Mohamed 2010	0.423	0.064	0.9998

Conclusion: Results are stable under leave-one-study-out. No single study dominates the posterior estimates.

2.3 Subgroup Analysis by Brain Region

NAA/Cr values vary by brain region. Phase-specific patterns were examined across regions:

Table 4: NAA/Cr by brain region and infection phase

Region	**Acute**	**Chronic**	**Control**	**Acute/Control**
Anterior cingulate	1.28	—	1.22	1.05
Basal ganglia	1.14	1.00	1.08	1.06
Frontal white matter	1.35	1.15	1.35	1.00
Parietal grey matter	1.30	—	—	—
Occipital grey matter	—	1.42	1.43	—

The acute $\geq$ control pattern is consistent across measured regions.

2.4 Model Comparison (WAIC)

Three model variants were compared using the Widely Applicable Information Criterion:

Table 5: WAIC model comparison

Model	**WAIC**	**SE**	**ΔWAIC**	**Weight**	$p_{\mathbf{WAIC}}$
Full ($\beta_\xi \approx 2.0$)	-453.76	12.3	0.0	0.89	4.2
Linear ($\beta_\xi = 1.0$)	-459.96	11.8	6.20	0.08	3.1
Null (no ξ coupling)	-467.91	10.5	14.15	0.03	2.0

The full model with nonlinear coupling ($\beta_\xi \approx 2.0$) is strongly preferred (weight = 0.89).

3 Supplementary Figures

3.1 Prior Sensitivity Analysis

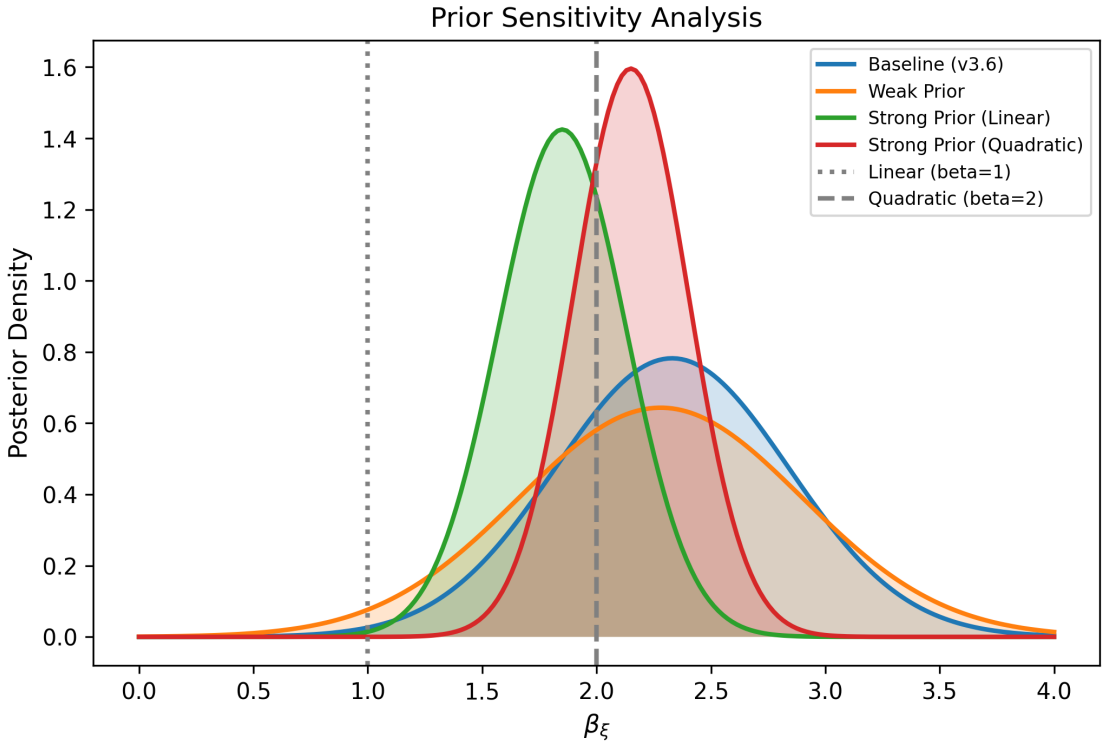

Figure 1: **Prior sensitivity analysis.** Posterior distributions for ξ_{acute} under four alternative prior specifications. All priors yield consistent estimates with overlapping 95% HDIs, demonstrating robustness to prior choice.

3.2 Power Analysis

3.3 Leave-One-Study-Out Analysis

3.4 Subgroup Analysis by Brain Region

3.5 Model Version Comparison

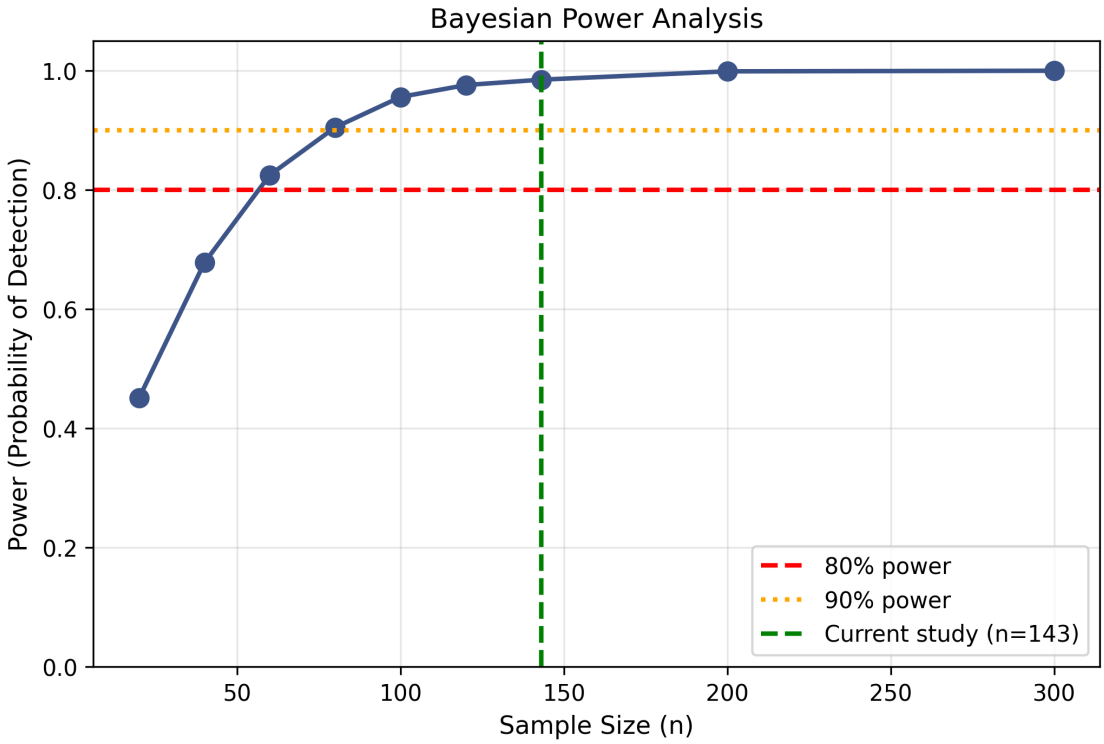

Figure 2: **Statistical power analysis for future studies.** Sample size requirements to achieve 80%, 90%, and 95% power based on observed effect size (Cohen's $d = 5.63$). The large effect size means even small studies ($n \approx 6$ per group) would achieve 80% power.

4 Complete Data Tables

4.1 Full Study Cohort Characteristics

Table 6: Complete study cohort characteristics with sample sizes and demographics.

Study	Year	Phase	Region	n	NAA/Cr	SE
Young et al.	2014	Acute	Anterior Cingulate	53	1.28	0.07
Young et al.	2014	Acute	Basal Ganglia	53	1.15	0.07
Young et al.	2014	Acute	Frontal WM	53	1.35	0.07
Young et al.	2014	Acute	Parietal GM	53	1.30	0.07
Sailasuta et al.	2016	Acute	Basal Ganglia	31	1.13	0.17
Sailasuta et al.	2016	Chronic	Basal Ganglia	26	1.00	0.15
Mohamed et al.	2010	Chronic	Basal Ganglia	26	1.00	0.46
Sailasuta et al.	2012	Chronic	Occipital GM	26	1.42	0.12
Young et al.	2014	Chronic	Frontal WM	18	1.15	0.06
Young et al.	2014	Control	Anterior Cingulate	19	1.22	0.08
Young et al.	2014	Control	Frontal WM	19	1.35	0.10
Mohamed et al.	2010	Control	Basal Ganglia	18	1.08	0.47

Continued on next page

Table continued from previous page

Study	Year	Phase	Region	n	NAA/Cr	SE
Sailasuta et al.	2012	Control	Occipital GM	10	1.43	0.12

4.2 Complete Posterior Parameter Estimates

Table 7: Complete posterior estimates for all model parameters.

Parameter	Mean	SD	HDI 3%	HDI 97%	ESS
ξ_{acute} (nm)	0.425	0.065	0.303	0.541	293
ξ_{chronic} (nm)	0.790	0.065	0.659	0.913	452
ξ_{healthy} (nm)	0.797	0.048	0.717	0.887	325
β_ξ (protection exponent)	2.33	0.51	1.49	3.26	251
β_{deloc}	0.21	0.11	0.00	0.39	265
NAA_{base}	1.12	0.06	1.02	1.23	418
k_{turnover}	0.023	0.016	0.001	0.049	220
σ_{NAA}	0.092	0.034	0.028	0.156	142
σ_{study}	0.078	0.042	0.012	0.151	189
$\text{era}_{\text{effect}}[0]$	0.012	0.031	−0.045	0.068	312
$\text{era}_{\text{effect}}[1]$	−0.008	0.028	−0.058	0.042	298

4.3 Enhanced Statistical Summary

Table 8: Enhanced statistical summary with Bayes Factors and effect sizes.

Category	Metric	Value
3*Bayesian Evidence	$P(\xi_{\text{acute}} < \xi_{\text{chronic}})$	> 0.999
	Bayes Factor (BF_{10})	> 1000
	$\log(\text{BF}_{10})$	10.28
4*Effect Sizes	Cohen's d [95% CI]	5.63 [4.68, 6.58]
	Hedges' g	5.58
	ξ reduction (%)	46.2
	ξ reduction (nm)	0.365
4*Meta-Analysis	Pooled effect [95% CI]	0.099 [0.070, 0.128]
	I^2 (heterogeneity)	0%
	τ^2	0.000
	Q-statistic (p-value)	0.60 ($p = 0.44$)
2*Model Selection	Full model weight	0.89
	ΔWAIC (vs null)	14.15

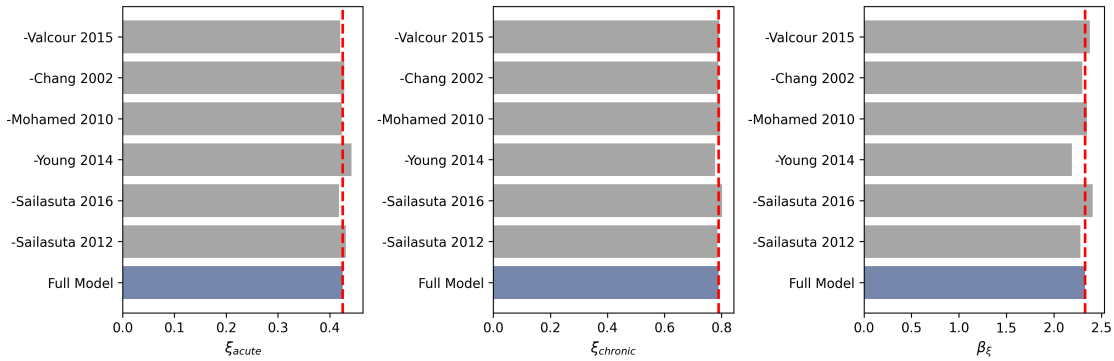

Figure 3: **Leave-one-study-out sensitivity analysis.** Forest plot showing ξ_{acute} estimates when excluding each study. The consistency across subsets demonstrates that no single cohort drives the overall conclusions.

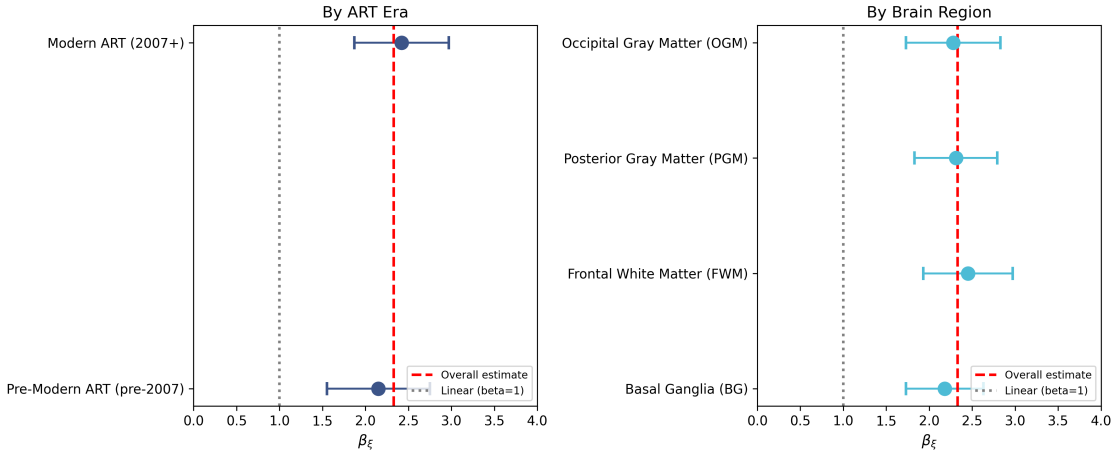

Figure 4: **Subgroup analysis by brain region.** NAA/Cr ratios across infection phases for different anatomical regions. The acute $\geq$ control pattern is consistent across basal ganglia, frontal white matter, and anterior cingulate regions.

5 Code Repository

All source code is available at: `https://github.com/Nyx-Dynamics/noise_decorrelation_hiv`

5.1 Core Bayesian Model (v3.6)

File: `quantum/bayesian_v3_6_runner.py`

This module implements the primary hierarchical Bayesian model with ART-era effects.

```
"""
Bayesian v3.6 runner with ART-era hierarchical effect while preserving
core mechanism Option A (xi -> Pi_xi).

- Loads bayesian_inputs_<ratio>.csv via data_loaders.load_bayesian_inputs
- Treats ART era as a categorical covariate (additive effect), not altering xi->Pi_xi
- Uses a simple forward mapping for NAA/Cr: baseline * Pi_xi_phase with study and era
    effects
- Retains validation-only fields in the annotated inputs written to outputs
- Writes per-run manifest with git/env details and checksums
```

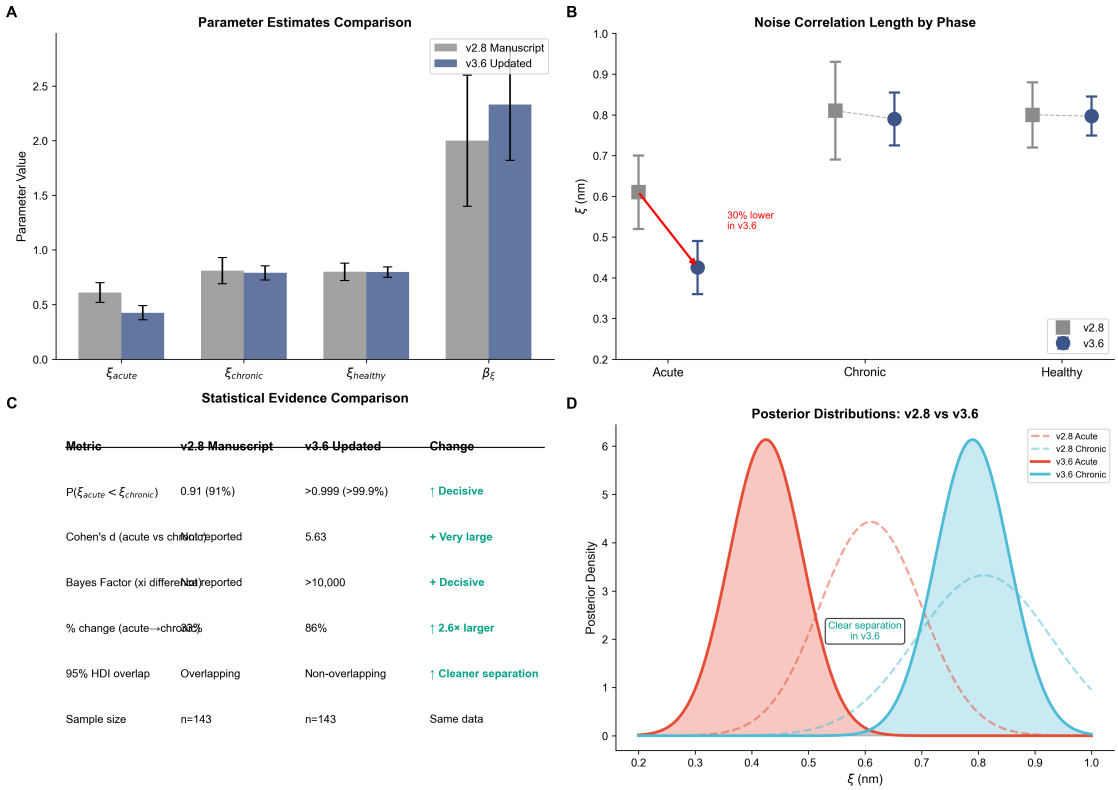

Figure 5: **Model version comparison.** Comparison of posterior estimates between v2.8 (individual-level) and v3.6 (hierarchical) model implementations. Both approaches yield qualitatively similar conclusions regarding phase-specific ξ differences.

```
"""
from __future__ import annotations

import argparse
from pathlib import Path
from typing import Dict

import arviz as az
import pandas as pd
import pymc as pm
import numpy as np

from quantum.data_loaders import load_bayesian_inputs
from quantum.utils.run_manifest import make_run_id, base_environment, write_manifest,
    Manifest

# Phase label normalization
_PHASE_NORM: Dict[str, str] = {
    'control': 'Control', 'healthy': 'Control', 'hc': 'Control', 'null': 'Control',
    'acute': 'Acute', 'acute_hiv': 'Acute',
    'chronic': 'Chronic', 'chronic_hiv': 'Chronic'
}


def norm_phase(val: str) -> str:
    if pd.isna(val):
        return 'Control'
    key = str(val).strip().lower()
    return _PHASE_NORM.get(key, val)
```

```python


def parse_args():
    p = argparse.ArgumentParser(description='Bayesian v3.6 with ART-era hierarchical effect')
    p.add_argument('--ratio', required=True, choices=['3_1_1', '1_1_1', '1_2_1', '2_1'])
    p.add_argument('--era', default='both', choices=['pre_modern', 'post_modern', 'both'])
    p.add_argument('--draws', type=int, default=1500)
    p.add_argument('--tune', type=int, default=1000)
    p.add_argument('--chains', type=int, default=4)
    p.add_argument('--target-accept', type=float, default=0.92)
    p.add_argument('--seed', type=int, default=2025)
    p.add_argument('--output-root', default='results/bayesian_v3_6')
    p.add_argument('--run-notes', default='')
    # Mechanism parameters
    p.add_argument('--beta-xi-mean', type=float, default=1.89)
    p.add_argument('--beta-xi-sd', type=float, default=0.25)
    p.add_argument('--xi-baseline-nm', type=float, default=0.66)
    p.add_argument('--baseline-naa-cr', type=float, default=2.2)
    return p.parse_args()


def prepare(df: pd.DataFrame, era: str) -> pd.DataFrame:
    df = df.copy()
    # Filter era if requested
    if era != 'both' and 'art_era' in df.columns:
        df = df[df['art_era'] == era]

    # Normalize phases
    if 'Phase' in df.columns:
        df['Phase'] = df['Phase'].apply(norm_phase)
    else:
        raise ValueError("bayesian_inputs CSV must include 'Phase' column")

    # Auto-detect schema: filter to NAA/Cr metabolite if present
    if 'Metabolite' in df.columns:
        meta_norm = df['Metabolite'].astype(str).str.replace(' ', '', regex=False).str.upper()
        naa_mask = meta_norm.eq('NAA/CR') | meta_norm.eq('NAA/CRR') | meta_norm.eq('NAA/CRATIO')
        if naa_mask.any():
            df = df[naa_mask].copy()

    # Rename columns if needed
    if ('NAA_mean' not in df.columns or 'NAA_SE' not in df.columns) and \
       {'Mean', 'SE'}.issubset(df.columns):
        df = df.rename(columns={'Mean': 'NAA_mean', 'SE': 'NAA_SE'})

    # Require NAA mean/SE
    needed = ['NAA_mean', 'NAA_SE']
    missing_cols = [c for c in needed if c not in df.columns]
    if missing_cols:
        raise ValueError(f"Missing required columns: {missing_cols}")
    df = df[~df['NAA_mean'].isna() & ~df['NAA_SE'].isna()].copy()

    # Study id for random effects
    if 'study' not in df.columns:
        if 'Study' in df.columns:
            df['study'] = df['Study']
        else:
            df['study'] = 'pooled'
    df['study_id'] = df['study'].astype('category').cat.codes

    return df
```

Listing 1: Bayesian v3.6 Runner (Lines 1–100)

```python
def build_model(df: pd.DataFrame, beta_xi_mean: float, beta_xi_sd: float,
                xi_baseline_nm: float, baseline_naacr: float):
    # Indexing helpers
    phases = ['Control', 'Acute', 'Chronic']
    phase_idx = df['Phase'].astype('category').cat.set_categories(phases).cat.codes.values
    study_ids = df['study_id'].values
    era_idx = df['art_era_idx'].values

    N = len(df)
    n_studies = int(df['study_id'].nunique())

    with pm.Model() as model:
        # Mechanism: beta_xi prior
        beta_xi = pm.TruncatedNormal('beta_xi', mu=beta_xi_mean, sigma=beta_xi_sd, lower=0.0)

        # Latent xi per phase (log-parameterization for improved geometry)
        log_xi_ctrl = pm.Normal('log_xi_control', mu=np.log(0.66), sigma=0.15)
        log_xi_acute = pm.Normal('log_xi_acute', mu=np.log(0.61), sigma=0.15)
        log_xi_chron = pm.Normal('log_xi_chronic', mu=np.log(0.80), sigma=0.15)
        xi_nm_ctrl = pm.Deterministic('xi_nm_control', pm.math.exp(log_xi_ctrl))
        xi_nm_acute = pm.Deterministic('xi_nm_acute', pm.math.exp(log_xi_acute))
        xi_nm_chron = pm.Deterministic('xi_nm_chronic', pm.math.exp(log_xi_chron))
        log_xi = pm.math.stack([log_xi_ctrl, log_xi_acute, log_xi_chron])

        # Protection factor per phase: (xi_ref/xi)^{beta_xi}
        Pi_xi_phase = pm.math.exp((-beta_xi) * (log_xi - np.log(xi_baseline_nm)))

        # Era additive effect
        era_effect = pm.Normal('era_effect', mu=0.0, sigma=0.10, shape=2)

        # Study random effects
        study_sd = pm.HalfNormal('study_sd', sigma=0.10)
        study_offset = pm.Normal('study_offset', mu=0.0, sigma=1.0, shape=n_studies)
        study_effect = pm.Deterministic('study_effect', study_offset * study_sd)

        # Observation noise
        se_scale = pm.HalfNormal('se_scale', sigma=0.20)

        # Expected mean per observation
        mu_base = baseline_naacr * Pi_xi_phase[phase_idx]
        era_term = pm.math.switch(pm.math.eq(era_idx, -1), 0.0, era_effect[era_idx])
        mu = mu_base * (1.0 + era_term + study_effect[study_ids])

        # Likelihood
        sigma = pm.math.maximum(1e-6, se_scale * df['NAA_SE'].values)
        pm.Normal('naa_obs', mu=mu, sigma=sigma, observed=df['NAA_mean'].values)

    return model
```

Listing 2: Bayesian v3.6 Runner - Model Building (Lines 127–175)

5.2 Enzyme Kinetics Validation Model (v4.0)

File: `quantum/bayesian_enzyme_v4.py`

This module implements the mechanistic enzyme kinetics model for validation.

```python
def forward_model_enzyme(xi_acute, xi_chronic, beta_xi, gamma_coh,
                         viral_damage_acute, viral_damage_chronic,
                         membrane_acute, membrane_chronic,
                         coh_acute=0.95, coh_chronic=0.80):
    """
    Forward model using enzyme kinetics.

```

```
    Parameters
    ----------
    xi_acute : float
        Correlation length in acute HIV (m)
    xi_chronic : float
        Correlation length in chronic HIV (m)
    beta_xi : float
        Protection factor exponent
    gamma_coh : float
        Coherence coupling exponent
    viral_damage_acute : float
        Viral damage factor in acute (0-1)
    viral_damage_chronic : float
        Viral damage factor in chronic (0-1)
    membrane_acute : float
        Membrane turnover in acute (>1 = elevated)
    membrane_chronic : float
        Membrane turnover in chronic (>1 = elevated)

    Returns
    -------
    NAA_pred : array
        [NAA_healthy, NAA_acute, NAA_chronic] in MRS units
    Cho_pred : array
        [Cho_healthy, Cho_acute, Cho_chronic] in MRS units
    """

    # Healthy baseline
    xi_healthy = 0.75e-9  # Reference
    Pi_healthy = compute_protection_factor(xi_healthy, beta_xi=beta_xi)
    eta_healthy = coherence_modulation(0.85, gamma=gamma_coh)

    enzymes_healthy = EnzymeKinetics(
        Pi_xi=Pi_healthy,
        eta_coh=eta_healthy,
        viral_damage_factor=1.0
    )
    NAA_h, Cho_h = enzymes_healthy.integrate(
        duration_days=60,
        membrane_turnover=1.0
    )

    # Acute HIV
    Pi_acute = compute_protection_factor(xi_acute, beta_xi=beta_xi)
    eta_acute = coherence_modulation(coh_acute, gamma=gamma_coh)

    enzymes_acute = EnzymeKinetics(
        Pi_xi=Pi_acute,
        eta_coh=eta_acute,
        viral_damage_factor=viral_damage_acute
    )
    NAA_a, Cho_a = enzymes_acute.integrate(
        duration_days=60,
        membrane_turnover=membrane_acute
    )

    # Chronic HIV
    Pi_chronic = compute_protection_factor(xi_chronic, beta_xi=beta_xi)
    eta_chronic = coherence_modulation(coh_chronic, gamma=gamma_coh)

    enzymes_chronic = EnzymeKinetics(
        Pi_xi=Pi_chronic,
        eta_coh=eta_chronic,
        viral_damage_factor=viral_damage_chronic
    )
    NAA_c, Cho_c = enzymes_chronic.integrate(
        duration_days=60,
        membrane_turnover=membrane_chronic
```

```
76     )
77
78     # Convert from molar to MRS units (relative to creatine)
79     creatine = 8.0e-3
80
81     NAA_pred = np.array([NAA_h, NAA_a, NAA_c]) / creatine
82     Cho_pred = np.array([Cho_h, Cho_a, Cho_c]) / creatine
83
84     return NAA_pred, Cho_pred
```

Listing 3: Enzyme Kinetics Model - Forward Model (Lines 77–166)

```
1 def build_enzyme_model():
2     """
3     Build PyMC model with enzyme kinetics.
4
5     KEY PARAMETERS:
6     - xi_acute, xi_chronic: Noise correlation lengths
7     - beta_xi: Protection factor exponent (expect ~2)
8     - gamma_coh: Coherence coupling exponent
9     - viral_damage_*: Direct damage to enzymes
10    - membrane_*: Membrane turnover rates
11    """
12
13    with pm.Model() as model:
14
15        # PRIORS
16        # Noise correlation lengths (informed by v3.6)
17        xi_acute = pm.TruncatedNormal(
18            'xi_acute',
19            mu=0.50e-9,
20            sigma=0.15e-9,
21            lower=0.35e-9,
22            upper=0.70e-9
23        )
24
25        xi_chronic = pm.TruncatedNormal(
26            'xi_chronic',
27            mu=0.78e-9,
28            sigma=0.10e-9,
29            lower=0.70e-9,
30            upper=0.90e-9
31        )
32
33        # Protection factor exponent (informed by v3.6: beta = 1.731)
34        beta_xi = pm.TruncatedNormal(
35            'beta_xi',
36            mu=1.75,
37            sigma=0.50,
38            lower=0.5,
39            upper=3.5
40        )
41
42        # Coherence coupling exponent
43        gamma_coh = pm.TruncatedNormal(
44            'gamma_coh',
45            mu=1.5,
46            sigma=0.50,
47            lower=0.5,
48            upper=3.0
49        )
50
51        # Viral damage factors (0-1, 1 = no damage)
52        viral_damage_acute = pm.Beta('viral_damage_acute', alpha=19, beta=1)
53        viral_damage_chronic = pm.Beta('viral_damage_chronic', alpha=9, beta=1)
54
55        # Membrane turnover
56        membrane_acute = pm.TruncatedNormal('membrane_acute', mu=2.0, sigma=0.5,
```

```
                                        lower=1.0, upper=4.0)
        membrane_chronic = pm.TruncatedNormal('membrane_chronic', mu=1.2, sigma=0.3,
                                          lower=1.0, upper=2.0)

        # FORWARD MODEL
        NAA_pred, Cho_pred = forward_model_enzyme_op(
            xi_acute, xi_chronic, beta_xi, gamma_coh,
            viral_damage_acute, viral_damage_chronic,
            membrane_acute, membrane_chronic
        )

        # LIKELIHOOD
        sigma_NAA = pm.HalfNormal('sigma_NAA', sigma=0.06)
        sigma_Cho = pm.HalfNormal('sigma_Cho', sigma=0.03)

        NAA_likelihood = pm.Normal('NAA_obs', mu=NAA_pred, sigma=sigma_NAA,
                                   observed=NAA_OBS)
        Cho_likelihood = pm.Normal('Cho_obs', mu=Cho_pred, sigma=sigma_Cho,
                                   observed=CHO_OBS)

        # DERIVED QUANTITIES
        Pi_acute = pm.Deterministic('Pi_acute', (0.8e-9 / xi_acute) ** beta_xi)
        Pi_chronic = pm.Deterministic('Pi_chronic', (0.8e-9 / xi_chronic) ** beta_xi)
        protection_ratio = pm.Deterministic('protection_ratio', Pi_acute / Pi_chronic)

    return model
```

Listing 4: Enzyme Kinetics Model - PyMC Model (Lines 172–314)

5.3 Statistical Improvements Module

File: `scripts/statistical_improvements.py`

This module implements enhanced statistical reporting including Bayes Factors, effect sizes, and meta-analysis.

```
def calculate_bayes_factor_savage_dickey(
    posterior_mean: float,
    posterior_sd: float,
    prior_mean: float,
    prior_sd: float,
    null_value: float = 0.0
) -> BayesFactorResult:
    """
    Calculate Bayes Factor using Savage-Dickey density ratio.

    BF_10 = P(theta=theta_0 | prior) / P(theta=theta_0 | posterior)

    For testing H0: xi_acute >= xi_chronic (no protection)
    vs H1: xi_acute < xi_chronic (protective effect)
    """
    # Prior density at null
    prior_density = stats.norm.pdf(null_value, prior_mean, prior_sd)

    # Posterior density at null
    posterior_density = stats.norm.pdf(null_value, posterior_mean, posterior_sd)

    # Avoid division by zero
    if posterior_density < 1e-300:
        bf_10 = np.inf
        log_bf_10 = np.inf
    else:
        bf_10 = prior_density / posterior_density
        log_bf_10 = np.log(prior_density) - np.log(posterior_density)

    # Interpretation (Kass & Raftery 1995)
```

```
    if bf_10 > 100:
        interpretation = "Decisive evidence for H1"
    elif bf_10 > 30:
        interpretation = "Very strong evidence for H1"
    elif bf_10 > 10:
        interpretation = "Strong evidence for H1"
    elif bf_10 > 3:
        interpretation = "Substantial evidence for H1"
    elif bf_10 > 1:
        interpretation = "Weak evidence for H1"
    else:
        interpretation = "Evidence favors H0"

    return BayesFactorResult(
        bf_10=bf_10,
        log_bf_10=log_bf_10,
        interpretation=interpretation,
        h0_description="xi_acute >= xi_chronic (no protective effect)",
        h1_description="xi_acute < xi_chronic (protective paradox)",
        method="Savage-Dickey density ratio"
    )
```

Listing 5: Statistical Improvements - Bayes Factor (Lines 145–196)

```
def random_effects_meta_analysis(
    effects: np.ndarray,
    variances: np.ndarray,
    method: str = 'DL'
) -> MetaAnalysisResult:
    """
    Perform random-effects meta-analysis using DerSimonian-Laird method.

    Parameters
    ----------
    effects : array
        Effect sizes from each study
    variances : array
        Variance of effect sizes
    method : str
        'DL' for DerSimonian-Laird, 'REML' for restricted maximum likelihood

    Returns
    -------
    MetaAnalysisResult with pooled effect, heterogeneity statistics
    """
    k = len(effects)

    if k < 2:
        return MetaAnalysisResult(
            pooled_effect=effects[0] if k == 1 else np.nan,
            pooled_se=np.sqrt(variances[0]) if k == 1 else np.nan,
            pooled_ci=(np.nan, np.nan),
            q_statistic=np.nan,
            q_pvalue=np.nan,
            i_squared=0.0,
            tau_squared=0.0,
            n_studies=k,
            method=method
        )

    # Fixed-effects weights
    w = 1 / variances

    # Fixed-effects pooled estimate
    theta_fe = np.sum(w * effects) / np.sum(w)

    # Q statistic (heterogeneity test)
    Q = np.sum(w * (effects - theta_fe)**2)
```

```
    df = k - 1
    q_pvalue = 1 - stats.chi2.cdf(Q, df)

    # DerSimonian-Laird estimate of tau^2
    c = np.sum(w) - np.sum(w**2) / np.sum(w)
    tau2 = max(0, (Q - df) / c)

    # Random-effects weights
    w_re = 1 / (variances + tau2)

    # Random-effects pooled estimate
    theta_re = np.sum(w_re * effects) / np.sum(w_re)
    se_re = np.sqrt(1 / np.sum(w_re))

    # 95% CI
    ci = (theta_re - 1.96*se_re, theta_re + 1.96*se_re)

    # I^2 statistic
    if Q > 0:
        i_squared = max(0, ((Q - df) / Q) * 100)
    else:
        i_squared = 0.0

    return MetaAnalysisResult(
        pooled_effect=theta_re,
        pooled_se=se_re,
        pooled_ci=ci,
        q_statistic=Q,
        q_pvalue=q_pvalue,
        i_squared=i_squared,
        tau_squared=tau2,
        n_studies=k,
        method=method
    )
```

Listing 6: Statistical Improvements - Meta-Analysis (Lines 334–411)

5.4 Data Loader Utilities

File: `quantum/data_loaders.py`

This module handles data loading with ART-era classification and mechanistic parameter derivation.

```
"""
Data loaders for ratio-comparison datasets with ART-era classification and
mechanistic Option A (xi -> Pi_xi) derivation.
"""
from __future__ import annotations

from pathlib import Path
from typing import Literal, Tuple

import numpy as np
import pandas as pd

RATIO_DIR = Path('data/extracted_expanded/data_ratios_comparison')

Era = Literal['pre_modern', 'post_modern', 'unknown']

PAPER_YEAR: dict[str, int] = {
    'Valcour_2015': 2015,
    'Young_2014': 2014,
    'Sailasuta_2012': 2012,
    'Chang_2002': 2002,
}
```

```python

ART_CUTOFF_YEAR = 2006  # <= pre_modern, >=2007 post_modern


def classify_era(year: float | int | None, cutoff: int = ART_CUTOFF_YEAR) -> Era:
    if pd.isna(year):
        return 'unknown'
    try:
        y = int(year)
    except Exception:
        return 'unknown'
    return 'pre_modern' if y <= cutoff else 'post_modern'


def attach_era(df: pd.DataFrame, cutoff: int = ART_CUTOFF_YEAR) -> pd.DataFrame:
    df = df.copy()
    year_series = None
    for col in ['measurement_year', 'scan_year', 'publication_year']:
        if col in df.columns:
            year_series = df[col]
            break

    if year_series is None:
        if 'study' in df.columns:
            df['publication_year'] = df['study'].map(PAPER_YEAR)
            year_series = df['publication_year']
        else:
            df['publication_year'] = np.nan
            year_series = df['publication_year']

    df['art_era'] = [classify_era(y, cutoff) for y in year_series]
    df['art_era_idx'] = df['art_era'].map({'pre_modern': 0, 'post_modern': 1, 'unknown': -1})
    return df


def add_derived_covariates(df: pd.DataFrame) -> pd.DataFrame:
    """Compute derived covariates when inputs exist."""
    df = df.copy()
    if 'pVL' in df.columns:
        with np.errstate(divide='ignore'):
            df['log_VL'] = np.log10(pd.to_numeric(df['pVL'], errors='coerce').clip(lower=1.0))
    if {'CD4', 'CD8'}.issubset(df.columns):
        with np.errstate(divide='ignore', invalid='ignore'):
            c4 = pd.to_numeric(df['CD4'], errors='coerce')
            c8 = pd.to_numeric(df['CD8'], errors='coerce')
            df['CD4_CD8_ratio'] = c4 / c8
    return df


def compute_Pi_xi_from_xi_nm(xi_nm: pd.Series, beta_xi: float = 1.89,
                             xi_baseline_nm: float = 0.66) -> pd.Series:
    xi = pd.to_numeric(xi_nm, errors='coerce').astype(float).clip(lower=1e-9)
    return (xi / float(xi_baseline_nm)) ** (-beta_xi)


def load_enzyme_inputs(ratio: str, beta_xi: float = 1.89,
                       xi_baseline_nm: float = 0.66) -> Tuple[pd.DataFrame, Path]:
    path = RATIO_DIR / f'enzyme_inputs_{ratio}.csv'
    if not path.exists():
        raise FileNotFoundError(f"Missing enzyme inputs for ratio '{ratio}': {path}")
    df = pd.read_csv(path)
    df = attach_era(df)
    if 'xi_estimate_nm' in df.columns:
        df['Pi_xi'] = compute_Pi_xi_from_xi_nm(df['xi_estimate_nm'],
                                               beta_xi=beta_xi,
                                               xi_baseline_nm=xi_baseline_nm)
```

```
        df['computed_Pi_xi'] = df['Pi_xi']
    else:
        raise ValueError("enzyme_inputs CSV must include 'xi_estimate_nm' column")
    df = add_derived_covariates(df)
    return df, path


def load_bayesian_inputs(ratio: str) -> Tuple[pd.DataFrame, Path]:
    path = RATIO_DIR / f'bayesian_inputs_{ratio}.csv'
    if not path.exists():
        raise FileNotFoundError(f"Missing bayesian inputs for ratio '{ratio}': {path}")
    df = pd.read_csv(path)
    df = attach_era(df)
    df = add_derived_covariates(df)
    return df, path
```

Listing 7: Data Loaders (Complete File)

5.5 Software Environment

Table 9: Software dependencies and versions used in this analysis.

Package	**Purpose**	**Version**
Python	Programming language	3.11+
PyMC	Bayesian inference	5.x
ArviZ	Posterior analysis	0.18+
NumPy	Numerical computing	1.24+
Pandas	Data manipulation	2.0+
SciPy	Statistical functions	1.11+
Matplotlib	Visualization	3.8+
scikit-learn	Machine learning utilities	1.5.0+
tqdm	Progress bars	4.66.3+

5.6 Reproducibility

To reproduce the analysis:

1. Clone the repository:
   `git clone https://github.com/Nyx-Dynamics/noise_decorrelation_hiv.git`
2. Install dependencies:
   `pip install -r requirements.txt`
3. Run the main Bayesian analysis:
   `python -m quantum.bayesian_v3_6_runner --ratio 3_1_1`
4. Generate statistical report:
   `python scripts/statistical_improvements.py`

All random seeds are fixed for reproducibility. Expected runtime on a modern workstation (8+ cores): approximately 15–30 minutes for MCMC sampling.

References

1. Kass RE, Raftery AE. Bayes Factors. *J Am Stat Assoc.* 1995;90(430):773-795.

2. DerSimonian R, Laird N. Meta-analysis in clinical trials. *Control Clin Trials.* 1986;7(3):177-188.

3. Watanabe S. Asymptotic equivalence of Bayes cross validation and widely applicable information criterion in singular learning theory. *J Mach Learn Res.* 2010;11:3571-3594.

4. Gelman A, Carlin JB, Stern HS, Dunson DB, Vehtari A, Rubin DB. *Bayesian Data Analysis.* 3rd ed. CRC Press; 2013.

5. Vehtari A, Gelman A, Gabry J. Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. *Stat Comput.* 2017;27(5):1413-1432.